

Inteligência Artificial

Estratégias de busca sem informação

Prof. Chauã Queirolo | <https://github.com/chaaua>

1

Sumário

Introdução

Tipos de agentes

Agentes de resolução de problemas

Problemas

2

Introdução

3

Agentes de resolução de problemas

- Esses **agentes** atuam em **ambientes** onde:
 - o objetivo é conhecido
 - as ações possíveis são conhecidas
 - o agente precisa planejar uma sequência de ações
- **Exemplo:**
 - encontrar a melhor jogada no xadrez
 - encontrar o melhor caminho em um mapa

4

Definição de problema

Estado inicial

situação inicial do agente

Ações

operações que podem ser executadas

Modelo de transição

resultado de aplicar uma ação em um estado

Teste de objetivo

verifica se o objetivo foi alcançado

Custo do caminho

custo total das ações executadas

Solução

Solução

Sequência de ações que levam do estado inicial ao estado objetivo

Solução ótima

Uma solução ótima é aquela com menor custo de caminho

Problema

7

Problema dos Magos e Zumbis

Em uma margem de um **rio** estão **três magos** e **três zumbis** que precisam atravessar para o outro lado usando um **barquinho** mágico

Sempre que houver **mais zumbis do que magos** em uma margem, eles tentam **morder os magos**



Regras da travessia

- O **barco** comporta no **máximo duas criaturas** por vez
- O **barco não** pode **atravessar vazio**
- Pelo menos uma criatura precisa pilotar o barco

Em nenhuma margem pode haver mais zumbis do que magos

Objetivo

Descobrir uma sequência de travessias que leve todos os magos e zumbis para a outra margem do rio sem que os zumbis tenham a chance de atacar

8

Problema dos Magos e Zumbis

Estado inicial

Modelo de transição

Ações

Teste de objetivo

Custo do caminho

9

Problema dos Magos e Zumbis

Estado inicial



Modelo de transição

Transportar 1 ou 2 personagens no 🚤 para a outra margem, atualizando suas quantidades na margem esquerda e invertendo a posição do barco, respeitando as restrições

Ações

Travessias permitidas no barco



Teste de objetivo



Custo do caminho

Número de travessias:

🚤 = 1

10

Problema dos Magos e Zumbis

Representação formal

$$s = (M, Z, B)$$

onde:

M = número de magos na margem esquerda

Z = número de zumbis na margem esquerda

B = posição do barco

- E = barco na margem esquerda
- D = barco na margem direita

Estado inicial

$$s = (3, 3, E)$$

Estado final

$$s = (0, 0, D)$$

Custo do caminho

$$c(s, a, s') = 1$$

Problema dos Magos e Zumbis

Ações

Como o barco leva no máximo 2 pessoas e no mínimo 1, as ações possíveis são:

- levar 1 mago $\rightarrow (1, 0)$
- levar 2 magos $\rightarrow (2, 0)$
- levar 1 zumbi $\rightarrow (0, 1)$
- levar 2 zumbis $\rightarrow (0, 2)$
- levar 1 mago e 1 zumbi $\rightarrow (1, 1)$

Essas ações valem tanto para ida quanto para volta; o que muda é o lado onde o barco está

Modelo de transição

Se o estado atual é $s = (M, Z, B)$ e a ação escolhida transporta (m, z) então:

Se o barco está na esquerda:
 $\text{resultado}((M, Z, B), (m, z)) = (M - m, Z - z, D)$

Se o barco está na direita:
 $\text{resultado}((M, Z, B), (m, z)) = (M + m, Z + z, E)$

Restrições:

1. Limite do barco: $1 \leq m + z \leq 2$

2. Segurança da margem esquerda:
 $M = 0$ ou $M \geq Z$

3. Segurança da margem direita:
 $(3 - M) = 0$ ou $(3 - M) \geq (3 - Z)$

Problema dos Magos e Zumbis

Estado inicial

$$s_0 = (3,3,E)$$

Ações

$$A = \{(1,0), (2,0), (0,1), (0,2), (1,1)\}$$

onde, $1 \leq m + z \leq 2$

Teste de objetivo

$$s_f = (0,0,D)$$

Modelo de transição

$$\text{Se } B = E: (M,Z,E) \rightarrow (M - m, Z - z, D)$$

$$\text{Se } B = D: (M,Z,D) \rightarrow (M + m, Z + z, E)$$

onde,

se $B = E$, $M=0$ ou $M \geq Z$

se $B = D$, $3-M = 0$ ou $3-M \geq 3-Z$

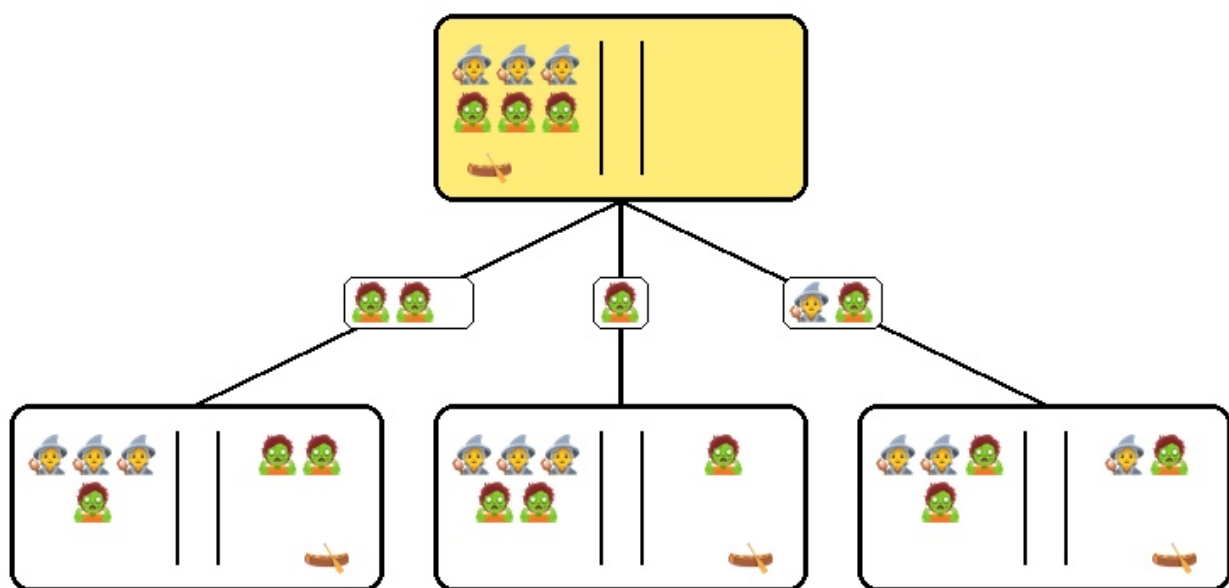
Custo do caminho

$$c(s,a,s') = 1$$

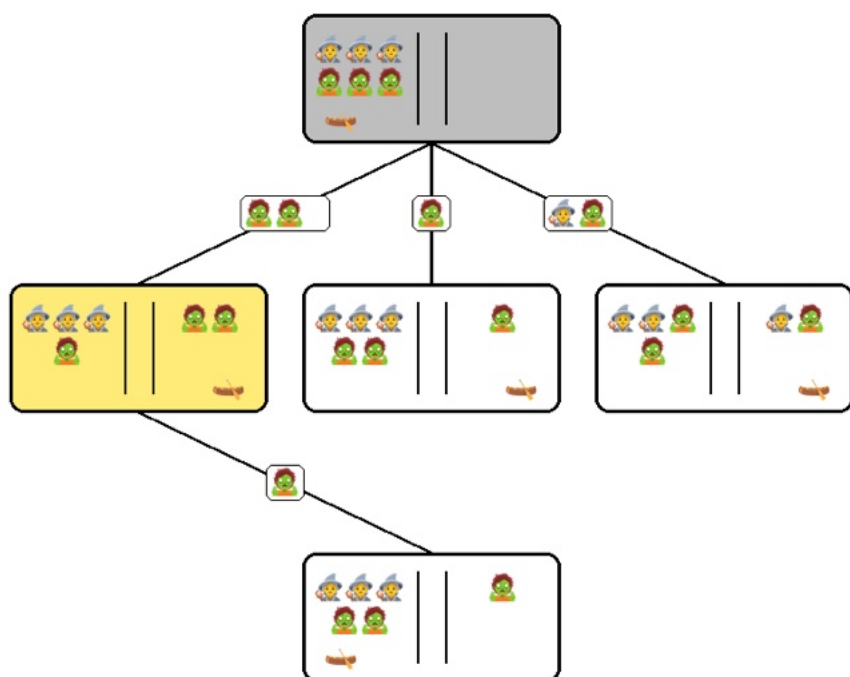
13



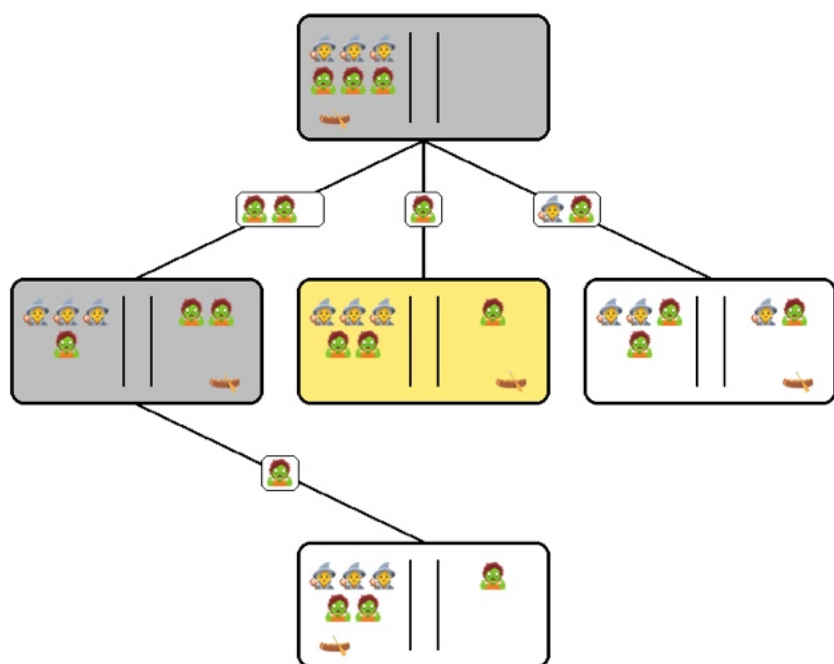
14-1



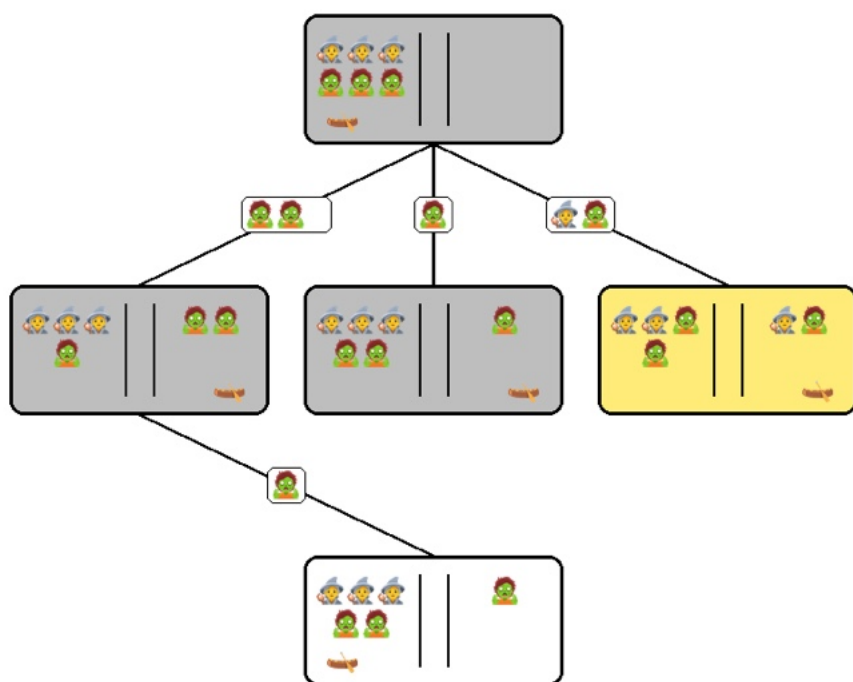
14-2



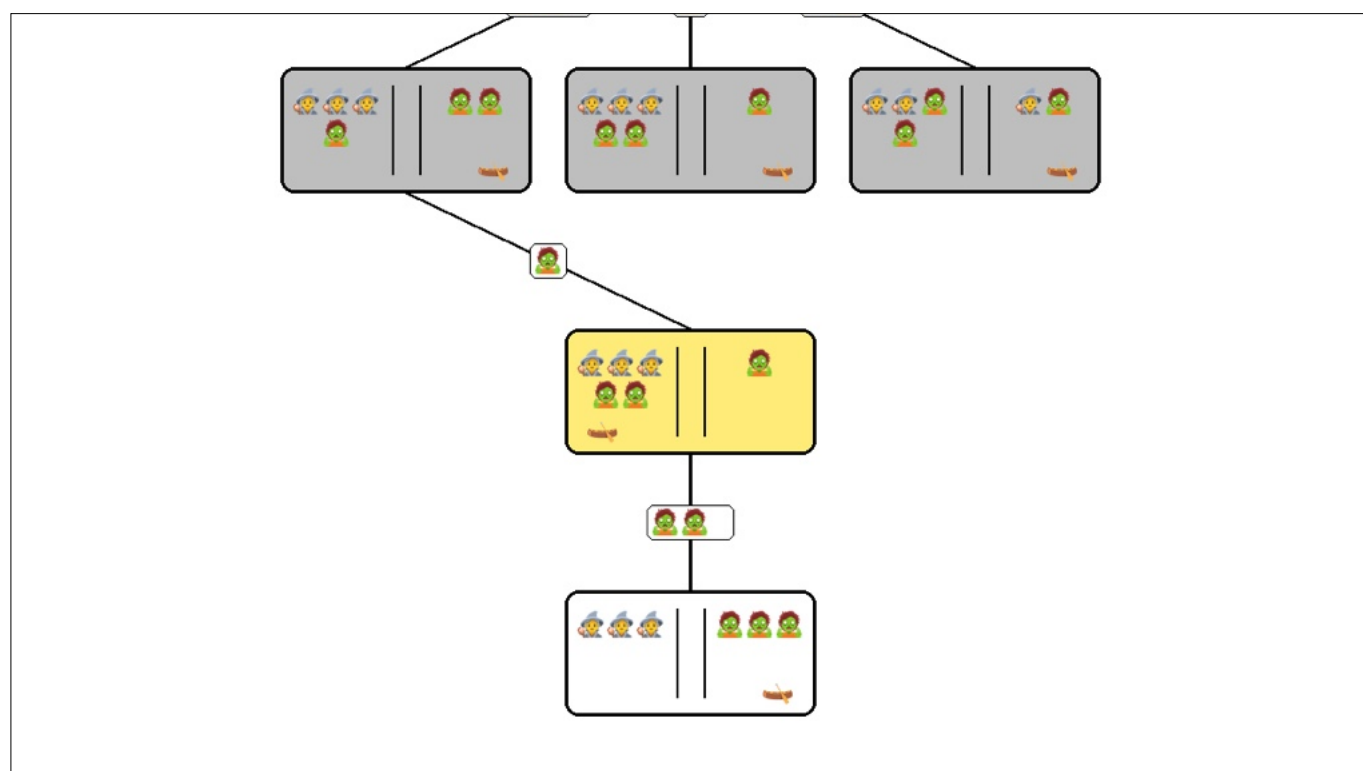
14-3



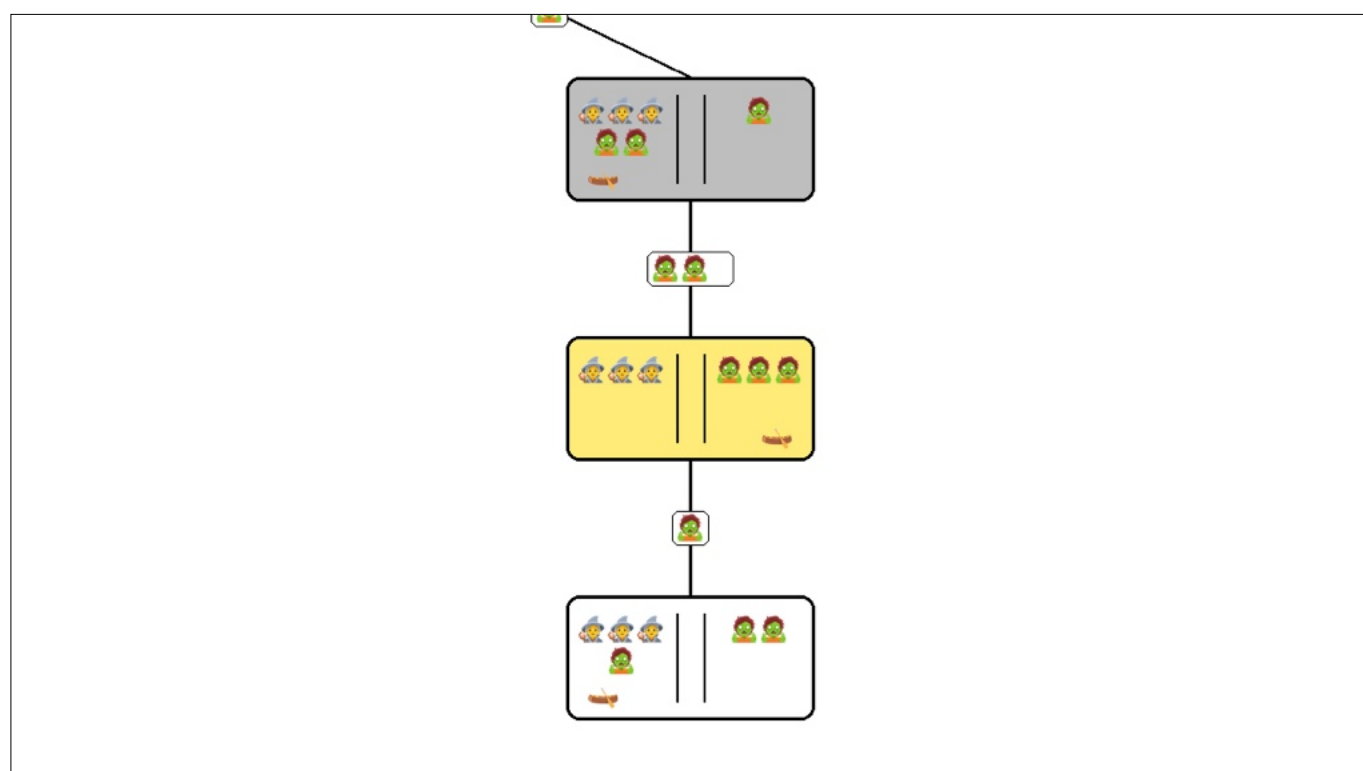
14-4



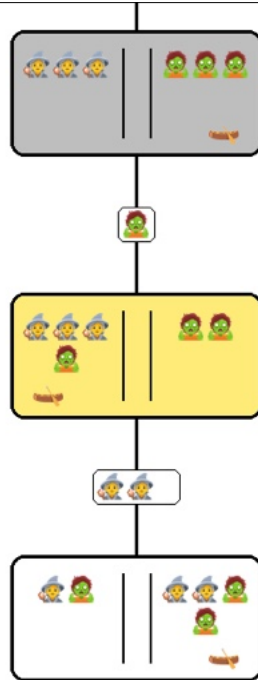
14-5



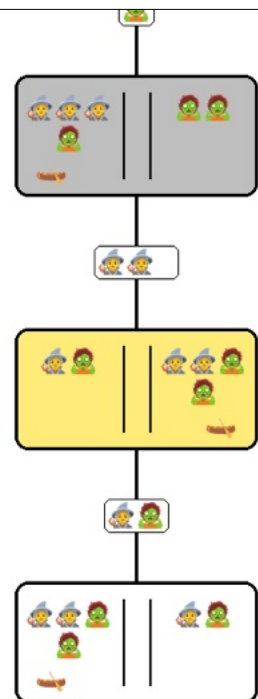
14-6



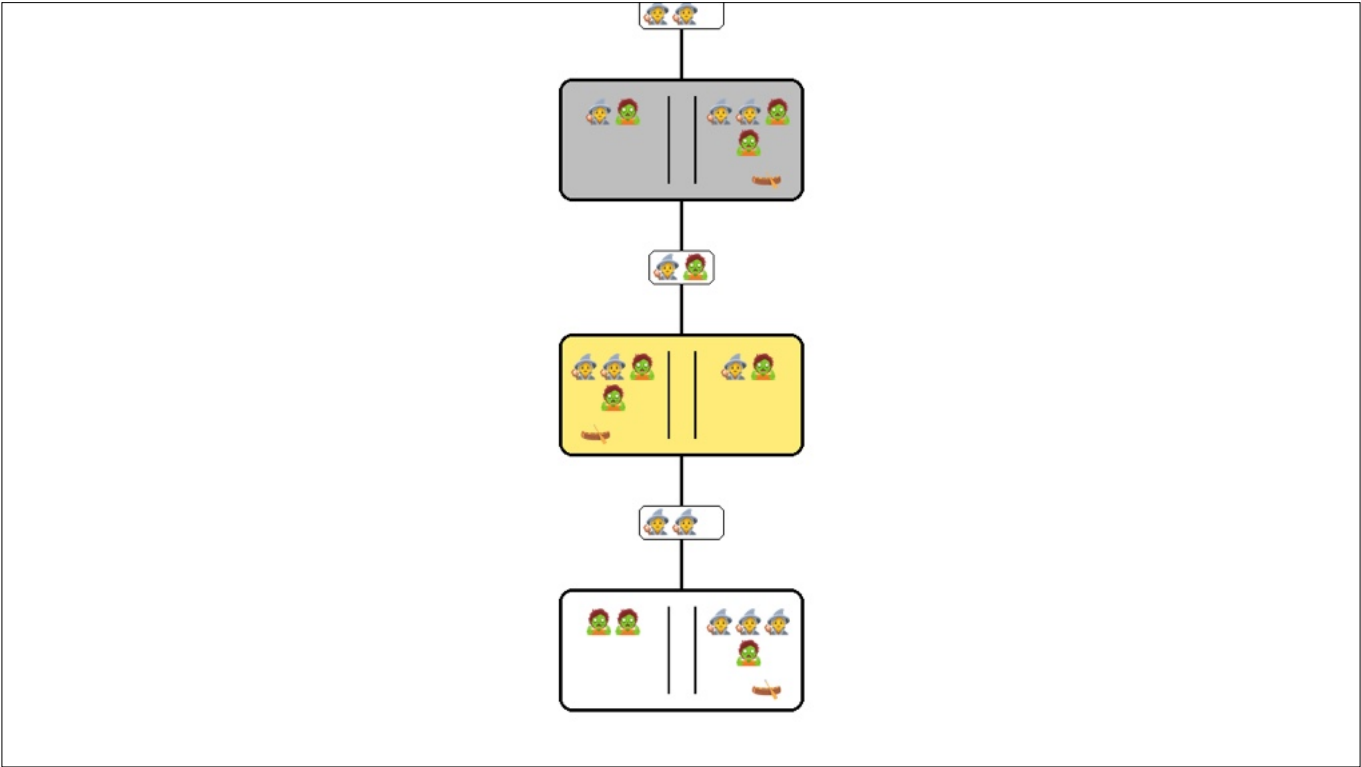
14-7



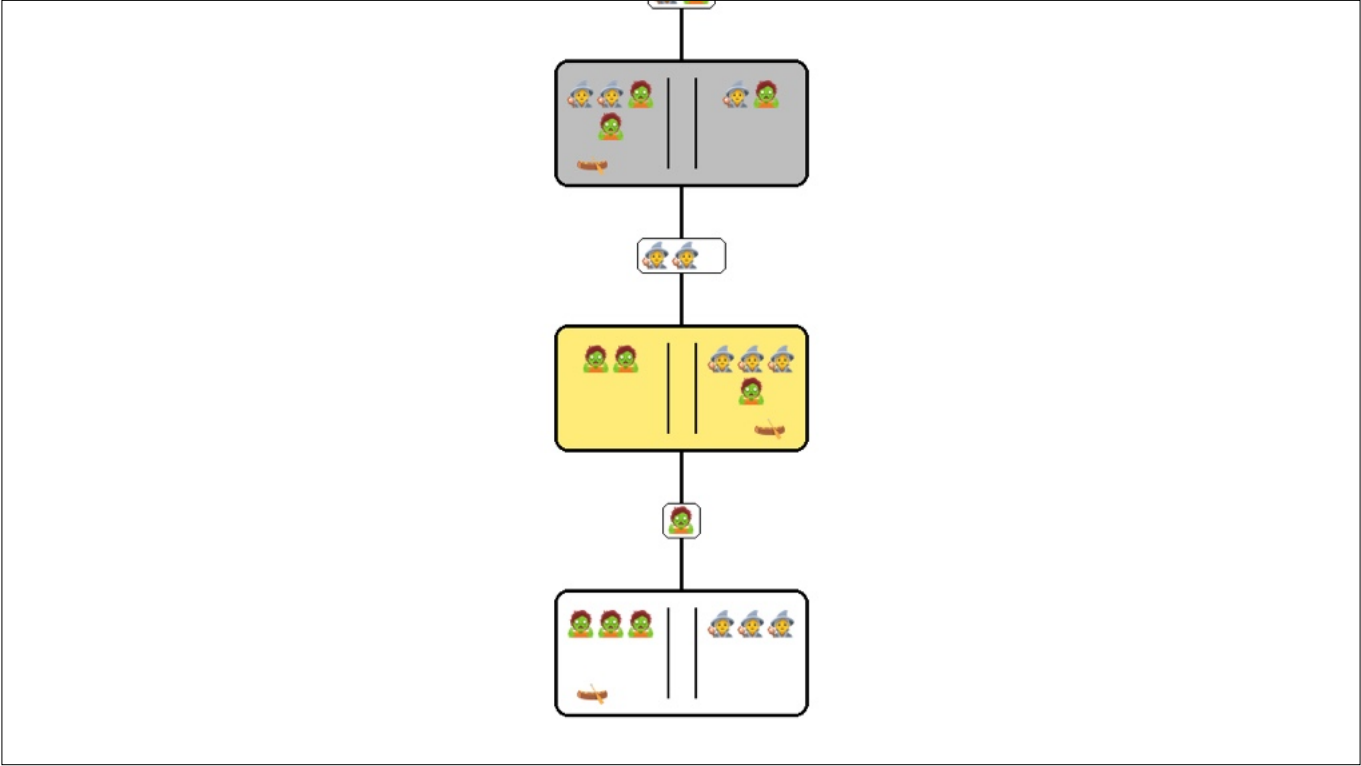
14-8



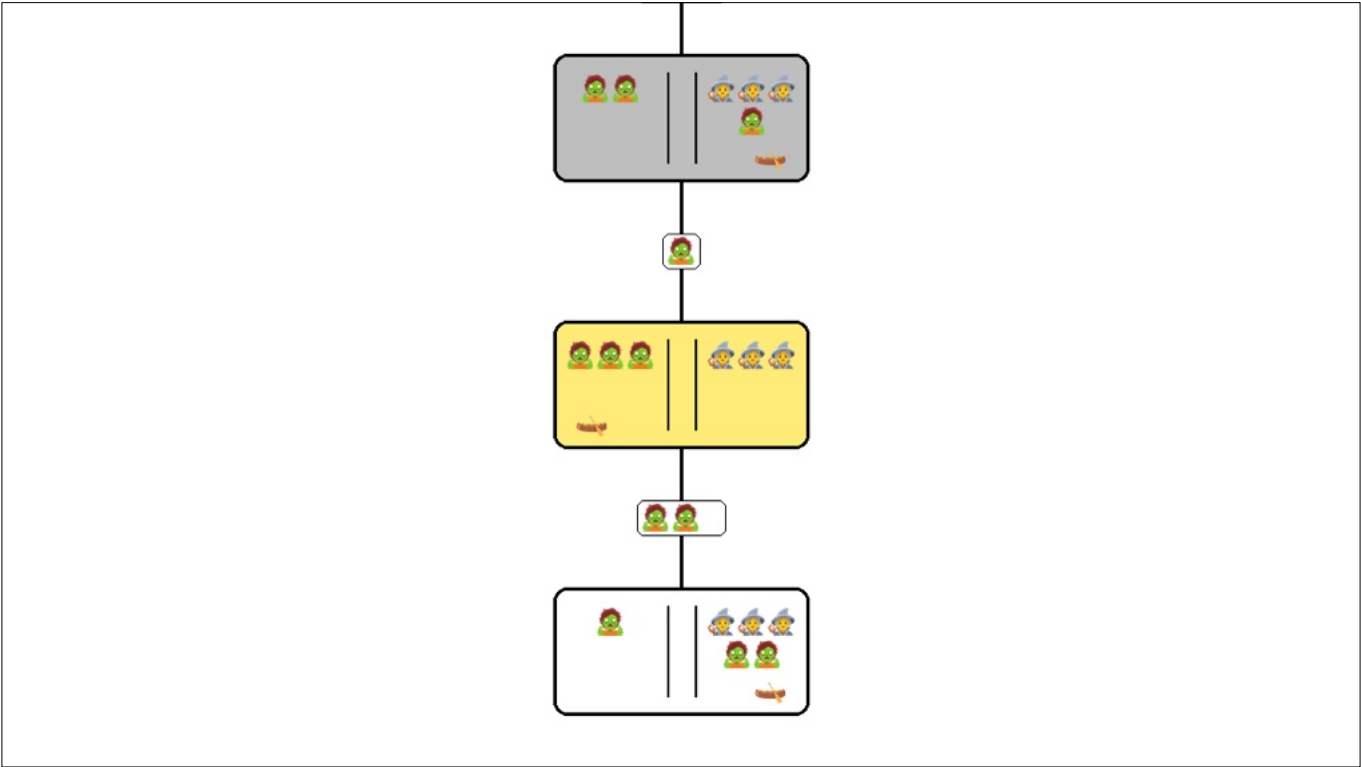
14-9



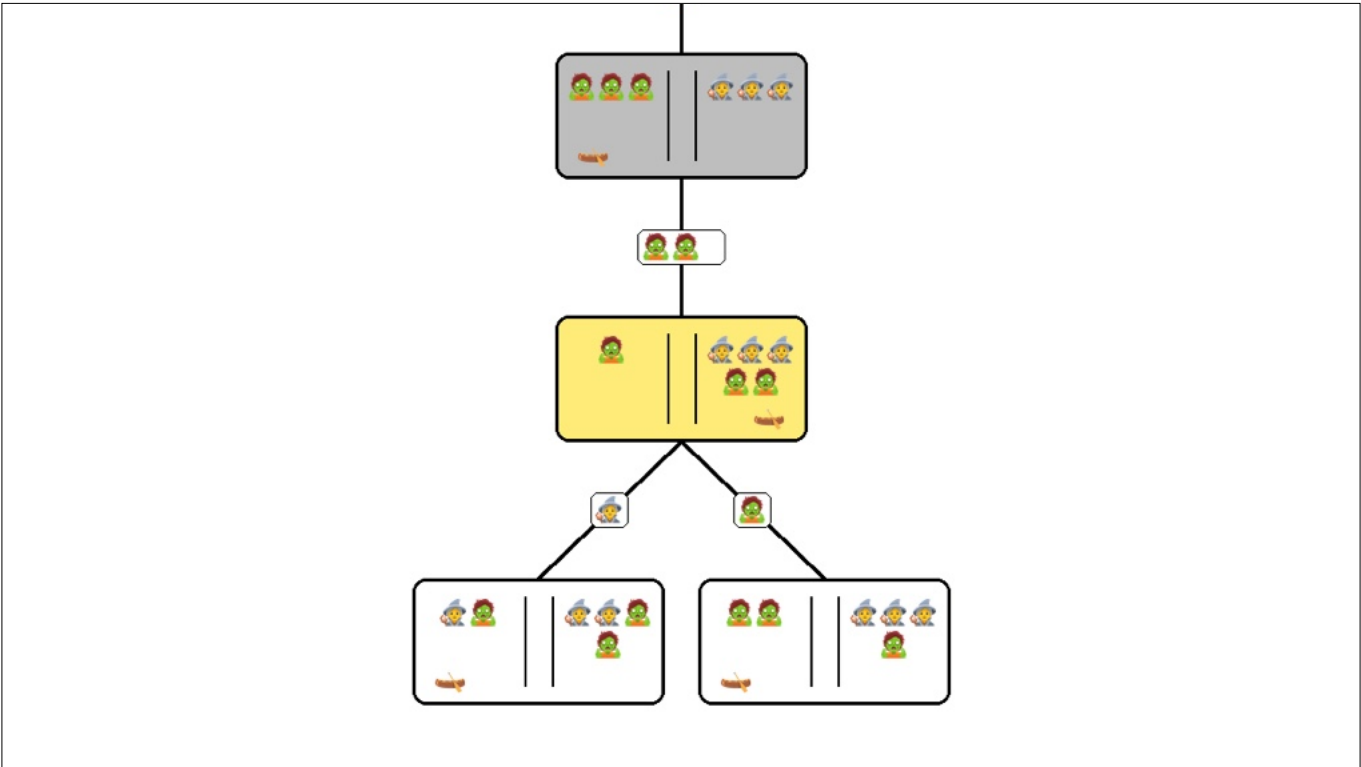
14-10



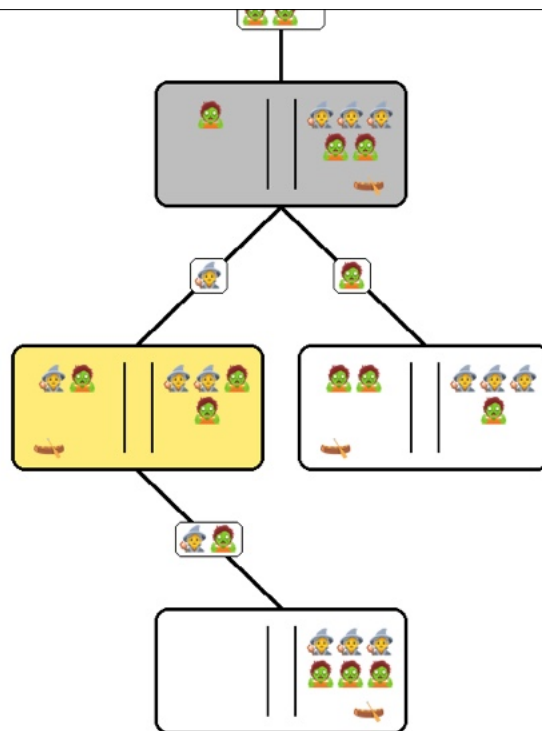
14-11



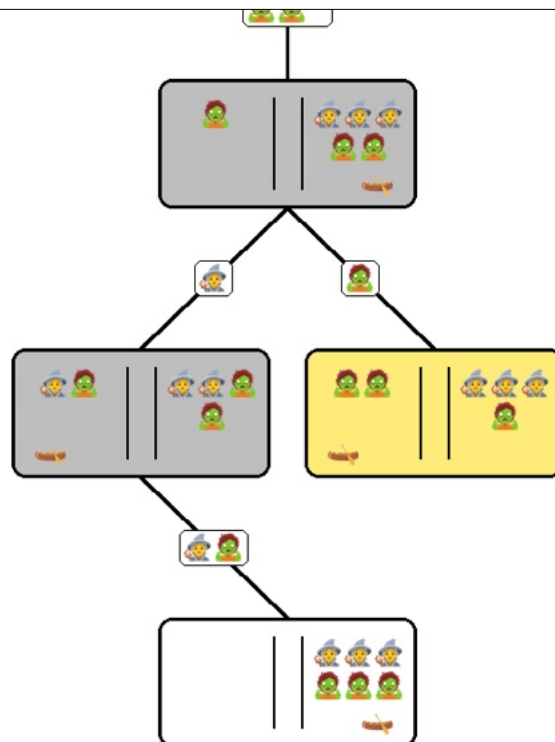
14-12



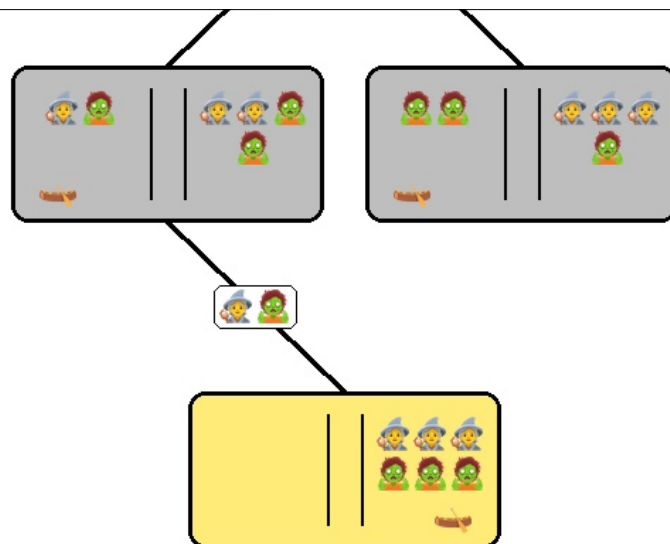
14-13



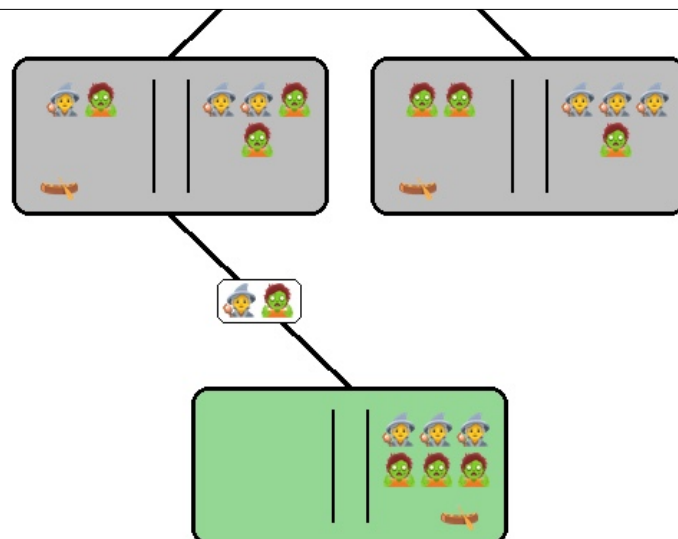
14-14



14-15



14-16



14-17

Quebra-cabeças de 8

Estado inicial

$S_0 =$

1	2	3
5	6	
4	7	8

Ações

$A = \{C, B, D, E\}$

Modelo de transição

Teste de objetivo

$S_f =$

1	2	3
4	5	6
7	8	

Custo do caminho

$c(s, a, s') = 1$

Busca sem informação

Busca sem informação

- Estratégias de **busca sem informação** usam apenas a **informação** disponível na definição do **problema**
- Apenas geram **sucessores** e verificam se o **estado objetivo** foi atingido

17

Algoritmo geral de busca em árvore

```
function TREE-SEARCH(problem, fringe) returns a solution, or failure
  fringe ← INSERT(MAKE-NODE(INITIAL-STATE[problem]), fringe)
  loop do
    if fringe is empty then return failure
    node ← REMOVE-FRONT(fringe)
    if GOAL-TEST[problem](STATE[node]) then return SOLUTION(node)
    fringe ← INSERT-ALL(EXPAND(node, problem), fringe)

function EXPAND(node, problem) returns a set of nodes
  successors ← the empty set
  for each action, result in SUCCESSOR-FN[problem](STATE[node]) do
    s ← a new NODE
    PARENT-NODE[s] ← node; ACTION[s] ← action; STATE[s] ← result
    PATH-COST[s] ← PATH-COST[node] + STEP-COST(node, action, s)
    DEPTH[s] ← DEPTH[node] + 1
    add s to successors
  return successors
```

18

Algoritmo geral de busca em árvore

```
Busca sem informação

1  BUSCA_EM_ÁRVORE(problema, fronteira)
2
3  fronteira ← [criar_nó(estado_inicial(problema))]
4
5  ENQUANTO fronteira ≠ ∅ FAÇA
6
7    nó ← remover_da_frenteira(fronteira)
8
9    SE teste_objetivo(estado(nó)) ENTÃO
10     RETORNAR solução(nó)
11
12    filhos ← expandir(nó, problema)
13    fronteira ← inserir_todos(filhos, fronteira)
14
15  RETORNAR falha
```

estado_inicial(problema)

Retorna o estado de partida do problema

criar_nó(estado)

Cria um nó com:

- estado
- pai = nulo
- custo = 0
- profundidade = 0

fronteira

Estrutura que armazena nós a explorar:

- fila → busca em largura
- pilha → busca em profundidade
- prioridade → busca informada

19-1

Algoritmo geral de busca em árvore

```
Busca sem informação

1  BUSCA_EM_ÁRVORE(problema, fronteira)
2
3  fronteira ← [criar_nó(estado_inicial(problema))]
4
5  ENQUANTO fronteira ≠ ∅ FAÇA
6
7    nó ← remover_da_frenteira(fronteira)
8
9    SE teste_objetivo(estado(nó)) ENTÃO
10     RETORNAR solução(nó)
11
12    filhos ← expandir(nó, problema)
13    fronteira ← inserir_todos(filhos, fronteira)
14
15  RETORNAR falha
```

estado_inicial(problema)

Retorna o estado de partida do problema

criar_nó(estado)

Cria um nó com:

- estado
- pai = nulo
- custo = 0
- profundidade = 0

fronteira

Estrutura que armazena nós a explorar:

- fila → busca em largura
- pilha → busca em profundidade
- prioridade → busca informada

19-2

Algoritmo geral de busca em árvore

```
Busca sem informação

1  BUSCA_EM_ÁRVORE(problema, fronteira)
2
3  fronteira ← [criar_nó(estado_inicial(problema))]
4
5  ENQUANTO fronteira ≠ ∅ FAÇA
6
7    nó ← remover_da_frenteira(fronteira)
8
9    SE teste_objetivo(estado(nó)) ENTÃO
10     RETORNAR solução(nó)
11
12    filhos ← expandir(nó, problema)
13    fronteira ← inserir_todos(filhos, fronteira)
14
15  RETORNAR falha
```

estado_inicial(problema)

Retorna o estado de partida do problema

criar_nó(estado)

Cria um nó com:

- estado
- pai = nulo
- custo = 0
- profundidade = 0

fronteira

Estrutura que armazena nós a explorar:

- fila → busca em largura
- pilha → busca em profundidade
- prioridade → busca informada

19-3

Algoritmo geral de busca em árvore

```
Busca sem informação

1  BUSCA_EM_ÁRVORE(problema, fronteira)
2
3  fronteira ← [criar_nó(estado_inicial(problema))]
4
5  ENQUANTO fronteira ≠ ∅ FAÇA
6
7    nó ← remover_da_frenteira(fronteira)
8
9    SE teste_objetivo(estado(nó)) ENTÃO
10     RETORNAR solução(nó)
11
12    filhos ← expandir(nó, problema)
13    fronteira ← inserir_todos(filhos, fronteira)
14
15  RETORNAR falha
```

estado_inicial(problema)

Retorna o estado de partida do problema

criar_nó(estado)

Cria um nó com:

- estado
- pai = nulo
- custo = 0
- profundidade = 0

fronteira

Estrutura que armazena nós a explorar:

- fila → busca em largura
- pilha → busca em profundidade
- prioridade → busca informada

19-4

Algoritmo geral de busca em árvore

```
Busca sem informação

1  BUSCA_EM_ÁRVORE(problema, fronteira)
2
3  fronteira ← [criar_nó(estado_inicial(problema))]
4
5  ENQUANTO fronteira ≠ ∅ FAÇA
6
7    nó ← remover_da_frenteira(fronteira)
8
9    SE teste_objetivo(estado(nó)) ENTÃO
10     RETORNAR solução(nó)
11
12    filhos ← expandir(nó, problema)
13    fronteira ← inserir_todos(filhos, fronteira)
14
15  RETORNAR falha
```

remover_da_frenteira(fronteira)

Remove um nó conforme estratégia de busca: FIFO, LIFO ou menor custo

teste_objetivo(estado)

Verifica se o estado é solução do problema

solução(nó)

Reconstrói o caminho da raiz até o nó:

- sequência de ações
- sequência de estados

20-1

Algoritmo geral de busca em árvore

```
Busca sem informação

1  BUSCA_EM_ÁRVORE(problema, fronteira)
2
3  fronteira ← [criar_nó(estado_inicial(problema))]
4
5  ENQUANTO fronteira ≠ ∅ FAÇA
6
7    nó ← remover_da_frenteira(fronteira)
8
9    SE teste_objetivo(estado(nó)) ENTÃO
10     RETORNAR solução(nó)
11
12    filhos ← expandir(nó, problema)
13    fronteira ← inserir_todos(filhos, fronteira)
14
15  RETORNAR falha
```

remover_da_frenteira(fronteira)

Remove um nó conforme estratégia de busca: FIFO, LIFO ou menor custo

teste_objetivo(estado)

Verifica se o estado é solução do problema

solução(nó)

Reconstrói o caminho da raiz até o nó:

- sequência de ações
- sequência de estados

20-2

Algoritmo geral de busca em árvore

```
Busca sem informação

1  BUSCA_EM_ÁRVORE(problema, fronteira)
2
3  fronteira ← [criar_nó(estado_inicial(problema))]
4
5  ENQUANTO fronteira ≠ ∅ FAÇA
6
7    nó ← remover_da_frenteira(fronteira)
8
9    SE teste_objetivo(estado(nó)) ENTÃO
10     RETORNAR solução(nó)
11
12    filhos ← expandir(nó, problema)
13    fronteira ← inserir_todos(filhos, fronteira)
14
15  RETORNAR falha
```

remover_da_frenteira(fronteira)

Remove um nó conforme estratégia de busca: FIFO, LIFO ou menor custo

teste_objetivo(estado)

Verifica se o estado é solução do problema

solução(nó)

Reconstrói o caminho da raiz até o nó:

- sequência de ações
- sequência de estados

20-3

Algoritmo geral de busca em árvore

```
Busca sem informação

1  BUSCA_EM_ÁRVORE(problema, fronteira)
2
3  fronteira ← [criar_nó(estado_inicial(problema))]
4
5  ENQUANTO fronteira ≠ ∅ FAÇA
6
7    nó ← remover_da_frenteira(fronteira)
8
9    SE teste_objetivo(estado(nó)) ENTÃO
10     RETORNAR solução(nó)
11
12    filhos ← expandir(nó, problema)
13    fronteira ← inserir_todos(filhos, fronteira)
14
15  RETORNAR falha
```

remover_da_frenteira(fronteira)

Remove um nó conforme estratégia de busca: FIFO, LIFO ou menor custo

teste_objetivo(estado)

Verifica se o estado é solução do problema

solução(nó)

Reconstrói o caminho da raiz até o nó:

- sequência de ações
- sequência de estados

20-4

Algoritmo geral de busca em árvore

```
Busca sem informação

1  BUSCA_EM_ÁRVORE(problema, fronteira)
2
3  fronteira ← [criar_nó(estado_inicial(problema))]
4
5  ENQUANTO fronteira ≠ ∅ FAÇA
6
7    nó ← remover_da_frenteira(fronteira)
8
9    SE teste_objetivo(estado(nó)) ENTÃO
10     RETORNAR solução(nó)
11
12    filhos ← expandir(nó, problema)
13    fronteira ← inserir_todos(filhos, fronteira)
14
15  RETORNAR falha
```

expandir(nó, problema)

Gera os filhos do nó:

- aplica ações possíveis
- cria novos nós
- atualiza custo e profundidade

inserir_todos(filhos, fronteira)

Adiciona vários nós na fronteira

21-1

Algoritmo geral de busca em árvore

```
Busca sem informação

1  BUSCA_EM_ÁRVORE(problema, fronteira)
2
3  fronteira ← [criar_nó(estado_inicial(problema))]
4
5  ENQUANTO fronteira ≠ ∅ FAÇA
6
7    nó ← remover_da_frenteira(fronteira)
8
9    SE teste_objetivo(estado(nó)) ENTÃO
10     RETORNAR solução(nó)
11
12    filhos ← expandir(nó, problema)
13    fronteira ← inserir_todos(filhos, fronteira)
14
15  RETORNAR falha
```

expandir(nó, problema)

Gera os filhos do nó:

- aplica ações possíveis
- cria novos nós
- atualiza custo e profundidade

inserir_todos(filhos, fronteira)

Adiciona vários nós na fronteira

21-2

Algoritmo geral de busca em árvore

```
Busca sem informação

1  BUSCA_EM_ÁRVORE(problema, fronteira)
2
3  fronteira ← [criar_nó(estado_inicial(problema))]
4
5  ENQUANTO fronteira ≠ ∅ FAÇA
6
7    nó ← remover_da_frenteira(fronteira)
8
9    SE teste_objetivo(estado(nó)) ENTÃO
10     RETORNAR solução(nó)
11
12    filhos ← expandir(nó, problema)
13    fronteira ← inserir_todos(filhos, fronteira)
14
15  RETORNAR falha
```

expandir(nó, problema)

Gera os filhos do nó:

- aplica ações possíveis
- cria novos nós
- atualiza custo e profundidade

inserir_todos(filhos, fronteira)

Adiciona vários nós na fronteira

21-3

Algoritmo geral de busca em árvore

```
Busca sem informação

1  EXPANDIR(nó, problema)
2
3  sucessores ← ∅
4
5  PARA CADA {ação, novo_estado} EM função_sucessora(problema, estado(nó)) FAÇA
6
7    filho ← criar_nó(novo_estado)
8
9    pai(filho) ← nó
10   ação(filho) ← ação
11   custo(filho) ← custo(nó) + custo_passo(nó, ação, novo_estado)
12   profundidade(filho) ← profundidade(nó) + 1
13
14   sucessores ← sucessores ∪ {filho}
15
16  RETORNAR sucessores
```

22

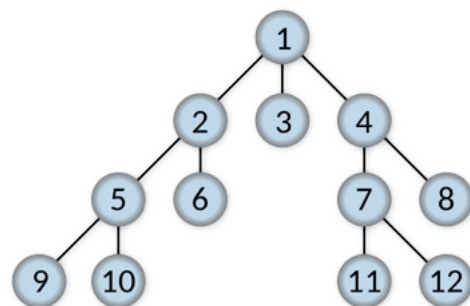
Busca sem informação

- As estratégias de busca sem informação se distinguem pela **ordem** em que os **nós** são **expandidos**
 - Busca em largura (Breadth-first)
 - Busca de custo uniforme
 - Busca em profundidade (Depth-first)
 - Busca em profundidade limitada
 - Busca de aprofundamento iterativo

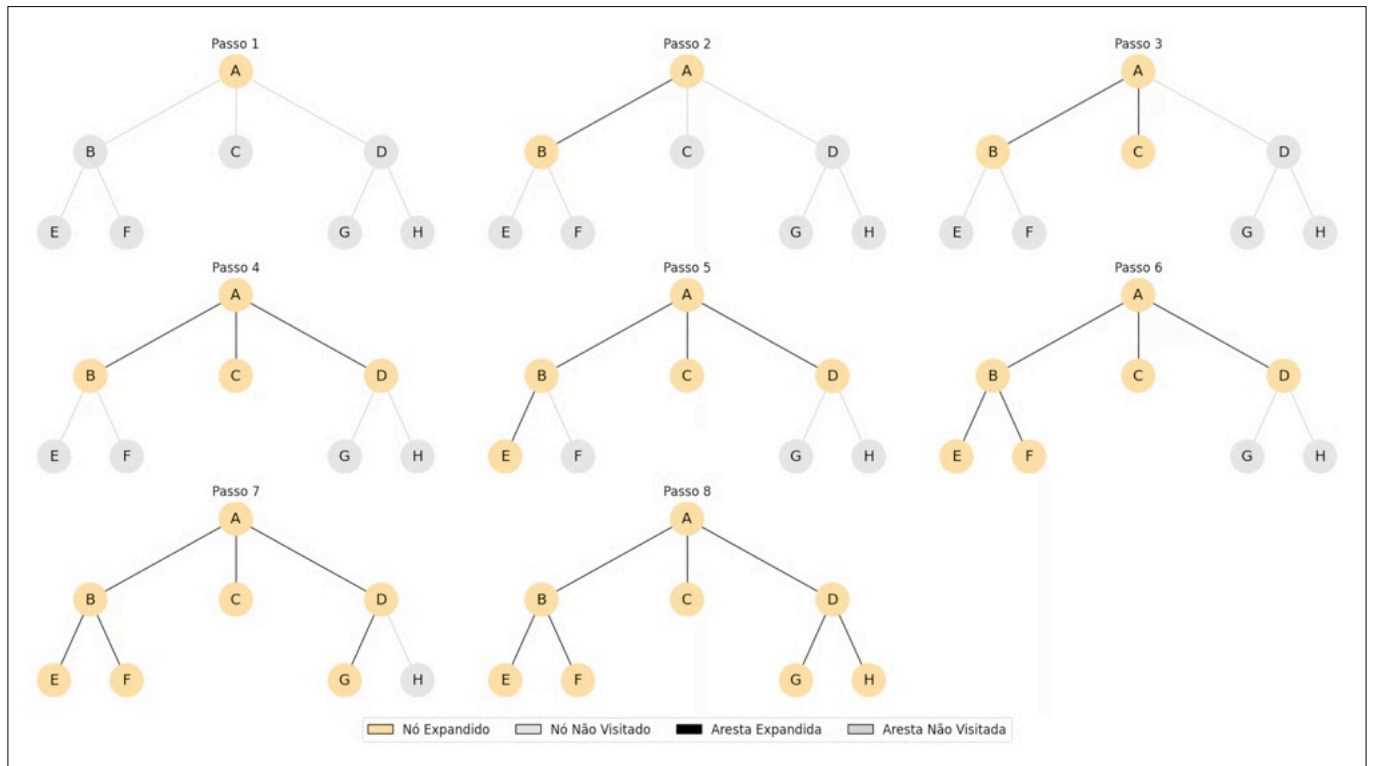
23

Busca em largura

- Expandir o nó não-expandido mais perto da raiz
- **Implementação**
 - A borda é uma fila FIFO
 - Novos itens entram no final.



24



25

Busca em largura

Completude

Sim, se o espaço de estados for finito

Otimalidade

Sim, se o custo de cada ação for igual

Complexidade de tempo: $O(b^d)$

Complexidade de espaço: $O(b^d)$

Onde:

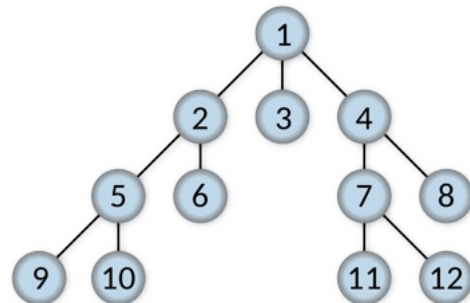
b = fator de ramificação

d = profundidade da solução mais rasa

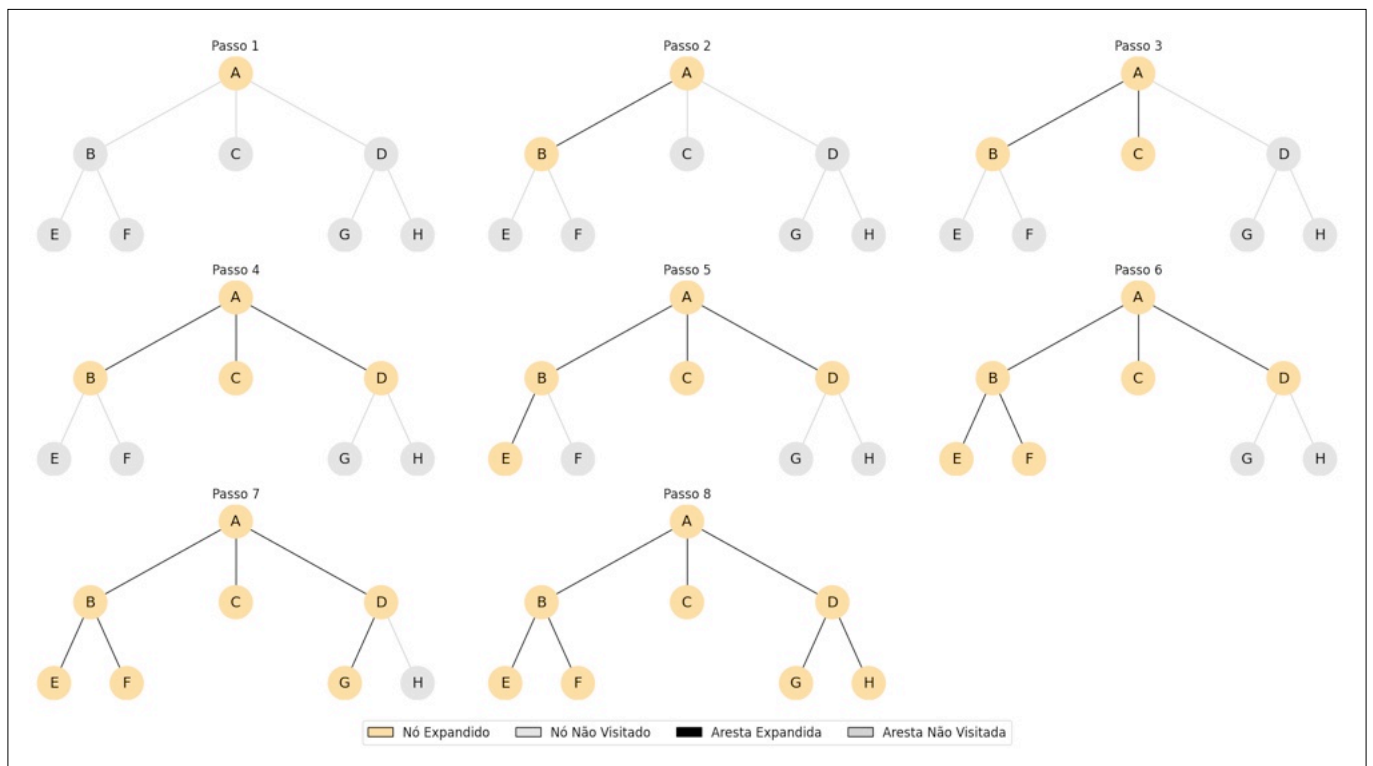
26

Busca em profundidade

- Expandir o nó não-expandido mais profundo
- **Implementação**
 - A borda é uma fila LIFO
 - Uma pilha



27



28

Busca em profundidade

Completude

Não, em espaços infinitos ou com ciclos

Otimidade

Não, pode retornar uma solução não ótima

Complexidade de tempo: $O(b^m)$

Complexidade de espaço: $O(b^m)$

Onde:

b = fator de ramificação

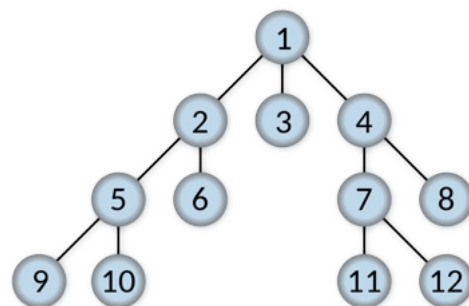
d = profundidade da solução mais rasa

m = profundidade máxima da árvore

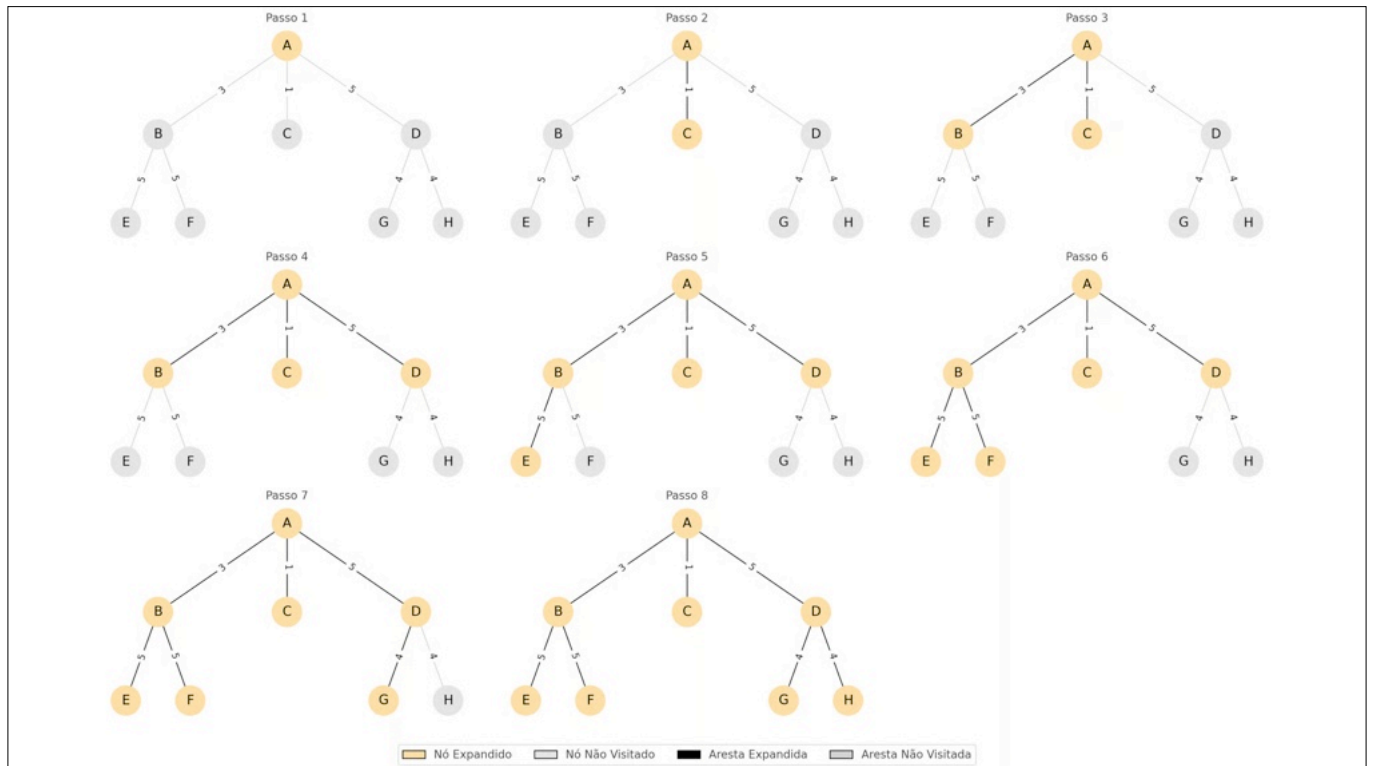
29

Busca de custo uniforme

- Expandir o nó de maior prioridade
- **Implementação**
 - A borda é uma fila HEAP
 - Uma fila de prioridades



30



31

Busca de custo uniforme

Completude

Sim, se os custos forem positivos

Otimalidade

Sim, pois sempre expande o caminho de menor custo

Complexidade de tempo: $O(b^{1+\lceil C^*/\epsilon \rceil})$

Complexidade de espaço: $O(b^{1+\lceil C^*/\epsilon \rceil})$

Onde:

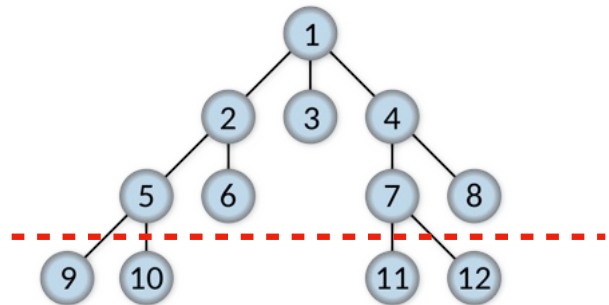
C^* = custo da solução ótima

ϵ = menor custo entre dois nós consecutivos

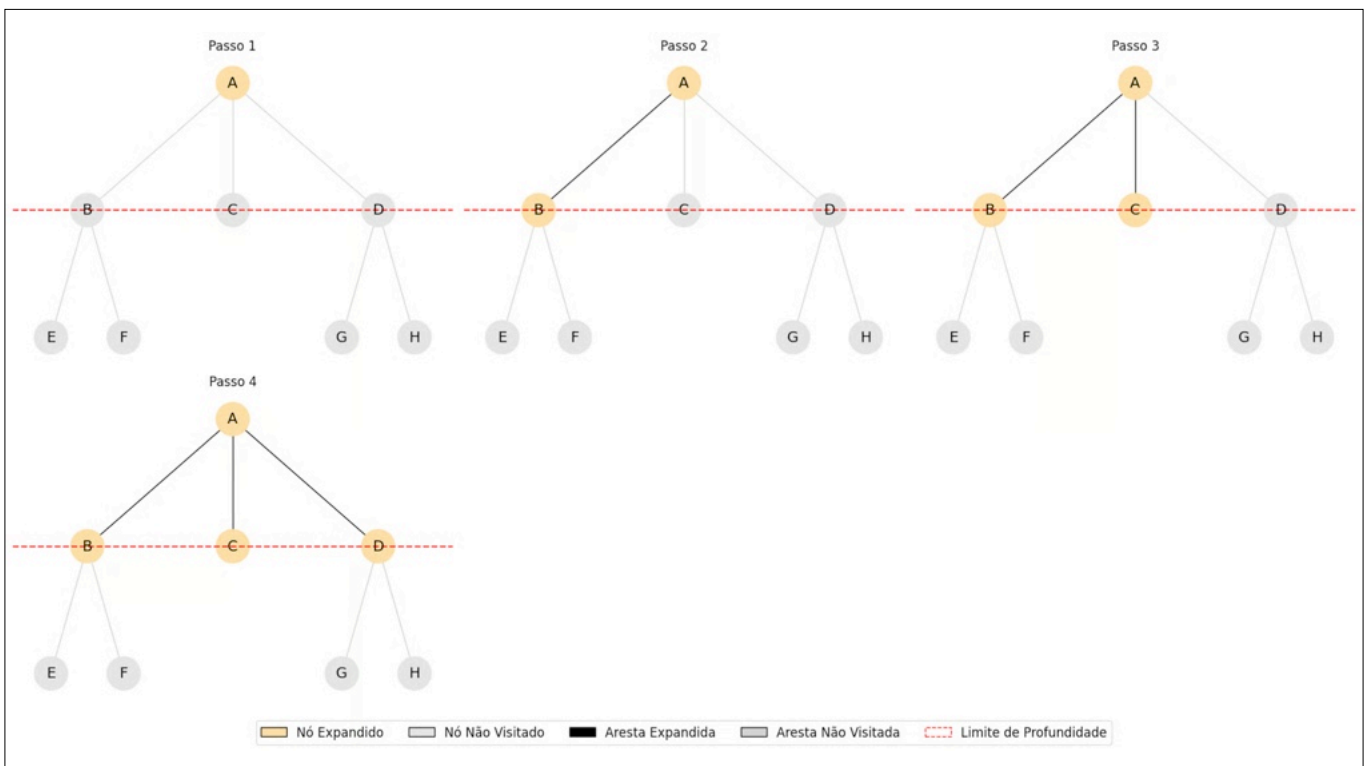
32

Busca em profundidade limitada

- Realiza busca em profundidade até uma altura definida



33



34

Busca em profundidade limitada

Completude

Sim, se $l \geq d$

Otimidade

Não, a menos que $l = d$ e todos os custos sejam iguais

Complexidade de tempo: $O(b^l)$

Complexidade de espaço: $O(b^l)$

Onde:

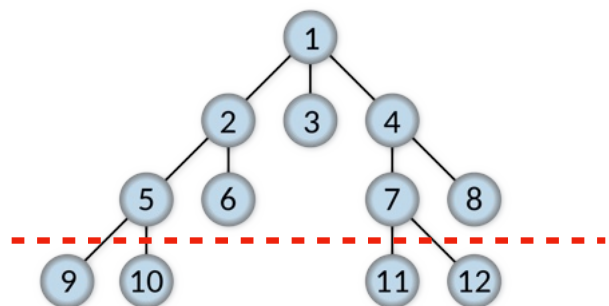
d = profundidade da solução

l = limite da árvore

35

Busca de aprofundamento iterativo

- Realiza busca em profundidade em níveis de altura pré-definidos



36

Busca em profundidade

Completeness

Sim, para espaços finitos

Optimality

Sim se o custo for uniforme

Complexidade de tempo: $O(b^d)$

Complexidade de espaço: $O(bd)$

37

Estados repetidos

- O **processo de busca** pode perder tempo expandindo nós já **explorados**
 - Estados repetidos podem levar a **loops infinitos**
 - Estados repetidos podem transformar um **problema linear** em um **problema exponencial**

38

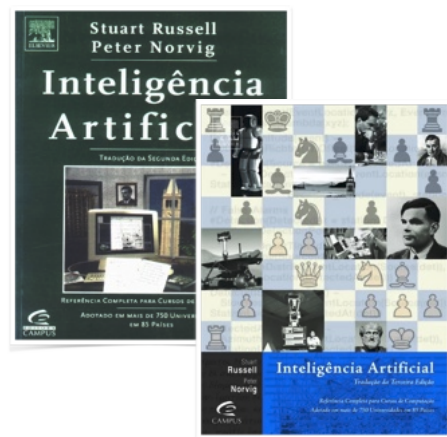
Estados repetidos

- **Comparar** os nós prestes a serem **expandidos** com nós já **visitados**.
 - Se o nó já tiver sido visitado, será descartado
 - Lista armazena nós já visitados
- Busca em profundidade e busca de aprofundamento iterativo não tem mais espaço linear
 - A busca percorre um grafo e não uma árvore

39

BIBLIOGRAFIA

- S. J. Russell & P. Norvig. **Artificial Intelligence: A Modern Approach**. Prentice Hall, 3rd edition, 2010.



40